

Ex.No:1

Experiment: Write a Python Program to Solve the 8-Puzzle Problem (Using BFS)

Aim

To develop a Python program that solves the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm and finds the shortest path from the initial state to the goal state.

Objectives

- To understand the concept of the 8-Puzzle problem.
- To implement the Breadth-First Search (BFS) algorithm in Python.
- To find the shortest sequence of moves from the initial state to the goal state.
- To display the solution path if it exists.
- To understand state-space search in Artificial Intelligence.

Algorithm

1. Start.
2. Read the initial state of the 8-puzzle.
3. Define the goal state.
4. Create a queue and insert the initial state.
5. Create a visited set to avoid revisiting states.
6. Remove the first state from the queue.
7. If the current state is the goal state, display the solution path and stop.
8. Otherwise, generate all possible next states by moving the blank tile (0) up, down, left, or right.
9. Add unvisited states to the queue and mark them as visited.
10. Repeat Steps 6–9 until the queue becomes empty.
11. If no solution is found, display "No Solution Exists."
12. Stop.

Program (Python)

```
from collections import deque
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)
moves = [(-1,0),(1,0),(0,-1),(0,1)]
def get_next_states(state):
    state = list(state)
    zero = state.index(0)
    row = zero // 3
    col = zero % 3
    next_states = []
    for dr, dc in moves:
        nr = row + dr
```

```

    nc = col + dc
    if 0 <= nr < 3 and 0 <= nc < 3:
        new = state[:]
        new_zero = nr * 3 + nc
        new[zero], new[new_zero] = new[new_zero], new[zero]
        next_states.append(tuple(new))
    return next_states
def bfs(start):
    queue = deque([(start, [])])
    visited = set()
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state not in visited:
            visited.add(state)
            for next_state in get_next_states(state):
                queue.append((next_state, path + [state]))
    return None
start = (1, 2, 3,
        4, 5, 6,
        7, 0, 8)
solution = bfs(start)
if solution:
    print("Solution Found!\n")
    for step in solution:
        for i in range(0, 9, 3):
            print(step[i:i+3])
    print()
else:
    print("No Solution Exists.")

```

Sample Input

Initial State

1 2 3

4 5 6

7 0 8

Goal State

1 2 3

4 5 6

7 8 0

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-01.py =====
Solution Found!
[(1, 2, 3),
(4, 5, 6),
(7, 0, 8)]
[(1, 2, 3),
(4, 5, 6),
(7, 8, 0)]
>>>
```

Result

The Python program was successfully implemented using the Breadth-First Search (BFS) algorithm to solve the 8-Puzzle problem. The program found the shortest sequence of moves from the initial state to the goal state and displayed the solution correctly.

Ex.No:2

Experiment: Write a Python Program to Solve the 8-Queen Problem

Aim

To develop a Python program that solves the 8-Queen problem using the Backtracking algorithm by placing eight queens on a chessboard such that no two queens attack each other.

Objectives

- To understand the 8-Queen problem.
- To implement the Backtracking algorithm in Python.
- To place eight queens safely on the chessboard.
- To ensure that no two queens share the same row, column, or diagonal.
- To display one valid solution.

Algorithm

1. Start.
2. Create an empty 8×8 chessboard.
3. Begin placing queens column by column.
4. For each column, check every row.
5. Verify whether placing a queen is safe by checking:

- Left row
 - Upper-left diagonal
 - Lower-left diagonal
6. If the position is safe, place the queen.
 7. Recursively place the queen in the next column.
 8. If no safe position exists, remove the previously placed queen (backtracking).
 9. Repeat until all eight queens are placed.
 10. Display the board.
 11. Stop.

Program (Python)

N = 8

Function to print the board

```
def print_board(board):
```

```
    for row in board:
```

```
        print(row)
```

Function to check if a queen can be placed

```
def is_safe(board, row, col):
```

```
    # Check left side of the row
```

```
    for i in range(col):
```

```
        if board[row][i] == 1:
```

```
            return False
```

```
    # Check upper-left diagonal
```

```
    i = row
```

```
    j = col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j] == 1:
```

```
            return False
```

```
        i -= 1
```

```

        j -= 1
    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1
    return True

# Backtracking function
def solve_queen(board, col):
    if col >= N:
        return True

    for row in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1

            if solve_queen(board, col + 1):
                return True

            # Backtrack
            board[row][col] = 0

    return False

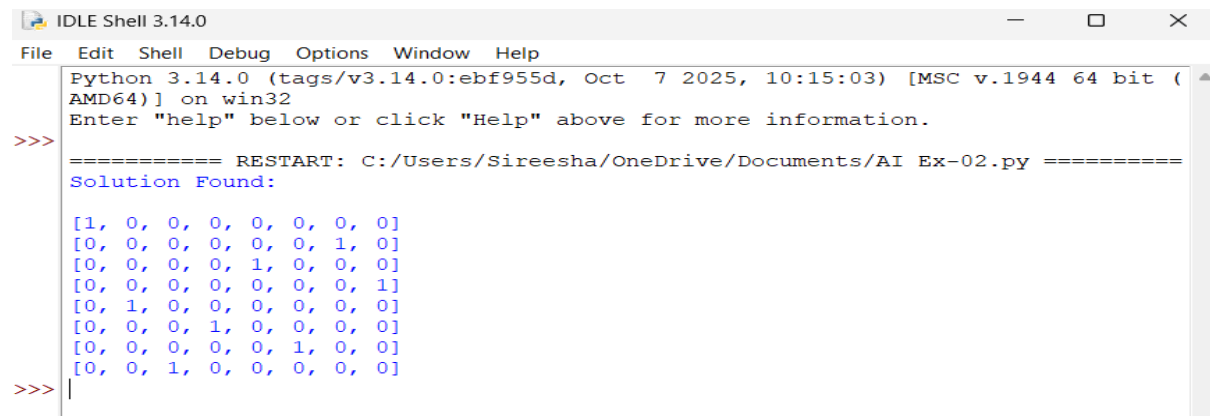
board = [[0 for _ in range(N)] for _ in range(N)]
if solve_queen(board, 0):
    print("Solution Found:\n")
    print_board(board)
else:
    print("No Solution Exists")

```

Sample Input

N = 8

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-02.py =====
Solution Found:
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
>>> |
```

Result

The Python program was successfully implemented using the Backtracking algorithm to solve the 8-Queen problem. The program placed all eight queens on the chessboard such that no two queens attacked each other and displayed a valid solution.

Ex.No:3

Experiment : Write a Python Program for the Water Jug Problem

Aim

The aim of this experiment is to write a Python program to solve the Water Jug Problem and obtain the required amount of water using two jugs by performing fill, empty, and transfer operations.

Objectives

- To understand the Water Jug problem.
- To implement the solution using Python.
- To perform filling, emptying, and transferring operations.
- To display the sequence of steps to reach the goal.

Algorithm

1. Start.
2. Initialize two jugs with their capacities.
3. Fill the first jug.
4. Pour water from the first jug to the second jug.
5. Empty the second jug when it becomes full.
6. Repeat the process until the required amount of water is obtained.
7. Display all the steps.
8. Stop.

Program (Python)

```
def water_jug(jug1, jug2, target):  
    x = 0  
    y = 0  
    print("Steps:")  
    while (x != target) and (y != target):  
        if x == 0:  
            x = jug1  
            print(x, y)  
        elif y == jug2:  
            y = 0  
            print(x, y)  
        else:  
            transfer = min(x, jug2 - y)  
            x -= transfer  
            y += transfer  
            print(x, y)  
    print("\nTarget Amount Reached!")
```

Capacity of jugs: 4L and 3L

water_jug(4, 3, 2)

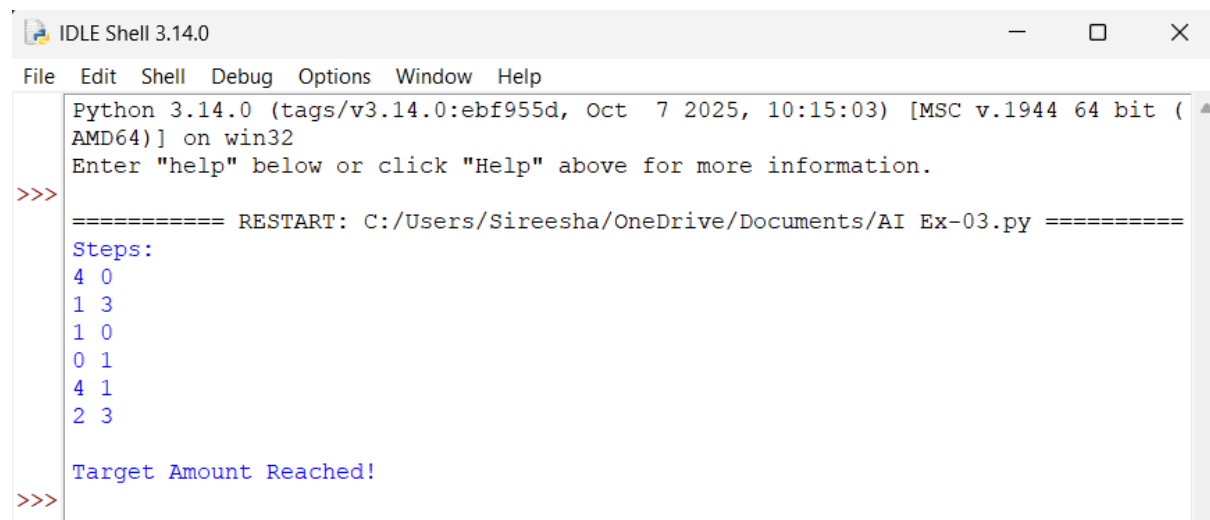
Sample Input

Jug 1 Capacity = 4 Litres

Jug 2 Capacity = 3 Litres

Target = 2 Litres

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-03.py =====
Steps:
4 0
1 3
1 0
0 1
4 1
2 3
Target Amount Reached!
>>>
```

Result

The Python program for the Water Jug Problem was successfully implemented. The program performed filling, emptying, and transferring operations to obtain the required amount of water (2 litres) and displayed the sequence of steps correctly.

Ex.No :4

Experiment : Write a Python Program for the Crypt-Arithmetic Problem

Aim

The aim of this experiment is to write a Python program to solve the Crypt-Arithmetic problem by assigning digits to letters such that the given arithmetic equation is satisfied.

Objectives

- To understand the Crypt-Arithmetic problem.
- To write a Python program to solve it.
- To assign unique digits to letters.
- To display the correct solution.

Algorithm

1. Start.
2. Read the given Crypt-Arithmetic expression.
3. Identify all unique letters.
4. Assign different digits (0–9) to each letter.
5. Check whether the arithmetic equation is satisfied.
6. If the equation is correct, display the solution.
7. Otherwise, try another combination.
8. Stop.

Program(Python)

```
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')

for p in permutations(range(10), 8):

    d = dict(zip(letters, p))

    # First letters cannot be zero
    if d['S'] == 0 or d['M'] == 0:
        continue

    send = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
    more = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
    money = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']

    if send + more == money:

        print("Solution Found\n")
```

```

print("SEND =", send)
print("MORE =", more)
print("MONEY =", money)
print("\nLetter Values:")
for k, v in d.items():
    print(k, "=", v)
break

```

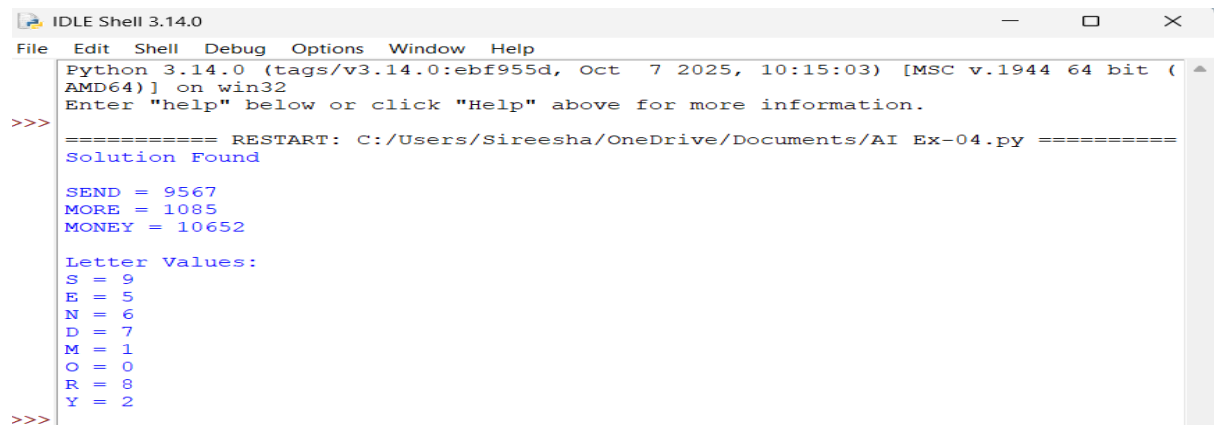
Sample Input

```

SEND
+ MORE
-----
MONEY

```

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-04.py =====
Solution Found
SEND = 9567
MORE = 1085
MONEY = 10652
Letter Values:
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2
>>>

```

Result

The Python program for the Crypt-Arithmetic Problem was successfully implemented. The program assigned unique digits to each letter and correctly solved the equation $SEND + MORE = MONEY$.

Ex.No :5

Experiment : Write a Python Program for the Missionaries and Cannibals Problem

Aim

The aim of this experiment is to write a Python program to solve the Missionaries and Cannibals problem by safely transporting all missionaries and cannibals across the river without violating the given conditions.

Objectives

- To understand the Missionaries and Cannibals problem.
- To write a Python program to solve the problem.
- To move all missionaries and cannibals safely across the river.
- To display the sequence of valid moves.

Algorithm

1. Start.
2. Initialize the number of missionaries and cannibals on both sides of the river.
3. Place the boat on the left side.
4. Generate all valid boat moves.
5. Check that missionaries are never outnumbered by cannibals on either side.
6. Move the boat and update the state.
7. Repeat until everyone reaches the right side.
8. Display the solution path.
9. Stop.

Program (Python)

```
from collections import deque  
  
def is_valid(m_left, c_left):  
    m_right = 3 - m_left
```

```

c_right = 3 - c_left
if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
    return False
if (m_left > 0 and c_left > m_left):
    return False
if (m_right > 0 and c_right > m_right):
    return False
return True
def solve():
    start = (3, 3, 'L')
    goal = (0, 0, 'R')
    moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
    queue = deque([(start, [start])])
    visited = set()
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        if state in visited:
            continue
        visited.add(state)
        m, c, boat = state
        for dm, dc in moves:
            if boat == 'L':
                new = (m-dm, c-dc, 'R')
            else:
                new = (m+dm, c+dc, 'L')
            if is_valid(new[0], new[1]):
                queue.append((new, path + [new]))

```

```
        return None

solution = solve()

print("Solution Path:\n")

for step in solution:

    print(step)
```

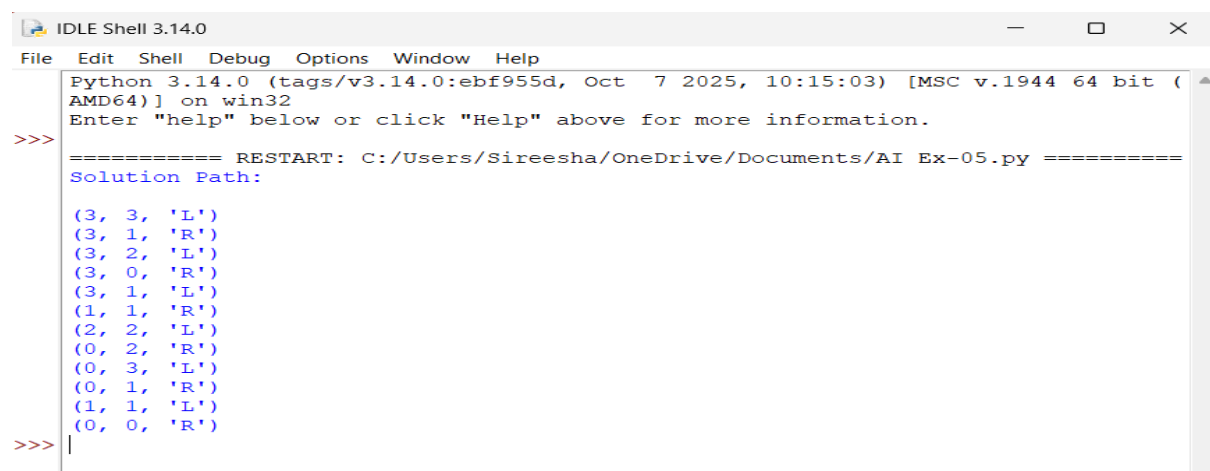
Sample Input

Missionaries = 3

Cannibals = 3

Boat Capacity = 2

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-05.py =====
Solution Path:
(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')
>>> |
```

Result

The Python program for the Missionaries and Cannibals Problem was successfully implemented. The program safely transported all 3 missionaries and 3 cannibals across the river while ensuring that the number of cannibals never exceeded the number of missionaries on either side. The sequence of valid moves was displayed successfully.

Ex.No :6

Experiment : Write a Python Program for the Vacuum Cleaner Problem

Aim

The aim of this experiment is to write a Python program to simulate the Vacuum Cleaner problem, where the vacuum cleaner cleans all dirty rooms by moving between them.

Objectives

- To understand the Vacuum Cleaner problem.
- To write a Python program to simulate the vacuum cleaner.
- To clean all dirty rooms.
- To display the actions performed by the vacuum cleaner.

Algorithm

1. Start.
2. Initialize the rooms and their status (Clean/Dirty).
3. Place the vacuum cleaner in the starting room.
4. Check whether the current room is dirty.
5. If the room is dirty, clean it.
6. Move to the next room.
7. Repeat until all rooms are clean.
8. Display the actions and final room status.
9. Stop.

Program (Python)

```
# Vacuum Cleaner Problem  
rooms = ['Dirty', 'Dirty']  
location = 0  
while location < len(rooms):
```

```

if rooms[location] == 'Dirty':
    print("Room", location + 1, "is Dirty")
    print("Cleaning Room", location + 1)
    rooms[location] = 'Clean'
else:
    print("Room", location + 1, "is already Clean")

location += 1

print("\nFinal Status of Rooms:")

for i in range(len(rooms)):
    print("Room", i + 1, ":", rooms[i])

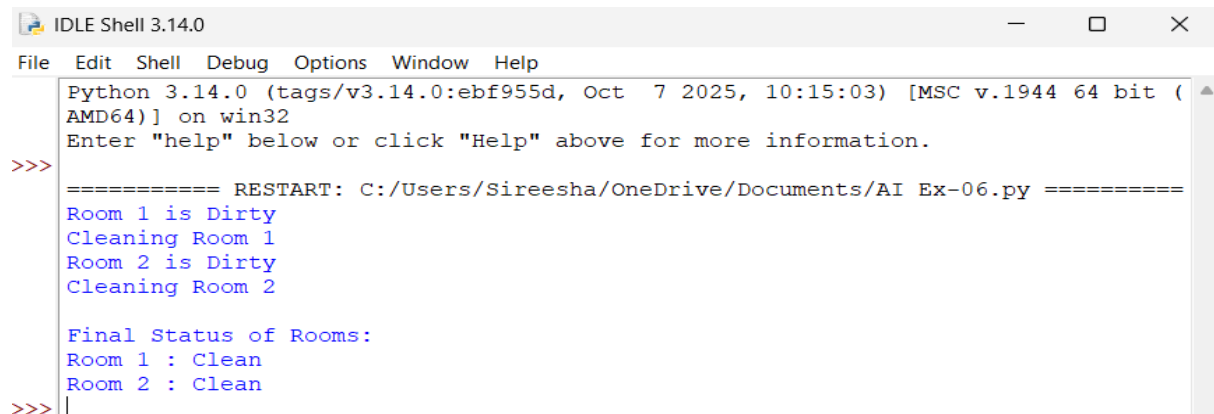
```

Sample Input

Room 1 = Dirty

Room 2 = Dirty

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-06.py =====
Room 1 is Dirty
Cleaning Room 1
Room 2 is Dirty
Cleaning Room 2

Final Status of Rooms:
Room 1 : Clean
Room 2 : Clean
>>>

```

Result

The Python program for the Vacuum Cleaner Problem was successfully implemented. The vacuum cleaner cleaned all the dirty rooms and displayed the final status of each room as Clean.

Ex.No :7

Experiment : Write a Python Program to Implement Breadth-First Search (BFS)

Aim

The aim of this experiment is to write a Python program to implement the Breadth-First Search (BFS) algorithm to traverse all the vertices of a graph.

Objectives

- To understand the BFS algorithm.
- To write a Python program for BFS.
- To traverse all vertices of a graph.
- To display the BFS traversal order.

Algorithm

1. Start.
2. Create a graph using an adjacency list.
3. Select the starting vertex.
4. Add the starting vertex to the queue.
5. Remove a vertex from the queue and display it.
6. Visit all unvisited adjacent vertices and add them to the queue.
7. Repeat until the queue is empty.
8. Stop.

Program (Python)

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],
```



```
'D': [],  
'E': ['F'],  
'F': []  
}  
visited = set()  
queue = deque()  
start = 'A'  
visited.add(start)  
queue.append(start)  
print("BFS Traversal:")  
while queue:  
    vertex = queue.popleft()  
    print(vertex, end=" ")  
    for neighbor in graph[vertex]:  
        if neighbor not in visited:  
            visited.add(neighbor)  
            queue.append(neighbor)
```

Sample Input

Graph:

$A \rightarrow B, C$

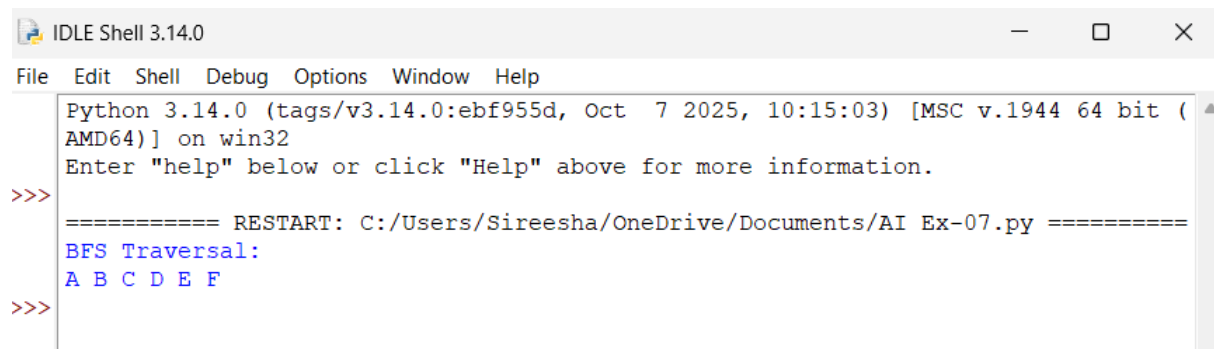
$B \rightarrow D, E$

$C \rightarrow F$

$E \rightarrow F$

Starting Vertex: A

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-07.py =====
BFS Traversal:
A B C D E F
>>>
```

Result

The Python program to implement the Breadth-First Search (BFS) algorithm was successfully executed. The graph was traversed level by level, and the BFS traversal order was displayed correctly.